Configuration and Report

After spending hours getting a T4, I was only able to use it for a few hours before I had difficulty starting it again due to google cloud resource constraints. Therefore, I followed a TA's advice (#213) to use a L4 instead, which was more widely available. My final configuration for all experiments:

Boot image: Google, Deep Learning VM with CUDA 11.3, M126, Debian 11, Python 3.10. With CUDA 11.3 preinstalled.

GPU: NVIDIA L4
CPU configuration: g2-standard-8 (8 vCPUs, 32 GB Memory)

Storage: 50GB

Reporting Methods:

As requested on Ed, kernel execution times only were timed, rather than including memory allocation and full end-to-end. For all kernel experiments except those in Part-C, a warmup was used. Data from single runs were used.

Part-A: CUDA Matrix Operations

Problem-1: Vector add and coalescing memory access

Q1

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd00 500
Total vector size: 3840000
Time: 0.000323 (sec), GFlopsS: 11.886441, GBytesS: 142.637290
Test PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd00 1000
Total vector size: 7680000
Time: 0.000875 (sec), GFlopsS: 8.777181, GBytesS: 105.326173
Test PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd00 2000
Total vector size: 15360000
Time: 0.001837 (sec), GFlopsS: 8.361390, GBytesS: 100.336679
Test PASSED
```

At first glance, I thought using a loop within a thread would be inefficient. However, having each thread add ValuesPerThread elements allows each thread to do enough work to warrant its overhead, known as thread coarsening. The results show an R^2 value of .9996,

indicating a strong linear correlation between the size of the vector and the execution time of the kernel. Both throughput, measured in GFlopS, and bandwidth, measured in GBytesS, decrease as the problem scales up. This indicates a problem with the implementation's memory access pattern. This program is especially sensitive to its memory management because of its low arithmetic intensity: 1 FLOP per 3 loads (4 bytes each) = 1/12 FLOPS/byte. The vecaddKernel00.cu does not use coalesced memory. For example, for ValuesPerThread=500, thread 0 works on A[0] - A[499], thread 1 works on A[500] - A[999], etc. This pattern prevents adjacent threads from loading adjacent data during the same iteration.

Q2

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd01 500
Total vector size: 3840000
Time: 0.000085 (sec), GFlopsS: 45.241931, GBytesS: 542.903169
Test PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd01 1000
Total vector size: 7680000
Time: 0.000506 (sec), GFlopsS: 15.180139, GBytesS: 182.161667
Test PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./vecadd01 2000
Total vector size: 15360000
Time: 0.001009 (sec), GFlopsS: 15.223183, GBytesS: 182.678193
Test PASSED
```

vecaddKernel01 outperforms vecaddKernel00 at every input value and across all metrics, execution time, throughput, and bandwidth. This was achieved by using a grid-stride by each thread. This ensures for a given iteration of the kernel (not a for-loop iteration), thread 0 loads data from index 0, thread 1 from index 1, as an example for iteration 0. This coalesced access allows DRAM to deliver data in a chunk, which is likely why the best bandwidth (542 GBytesS) was so much better than with vecaddKernel00. Although GflopsS and GBytesS both worsen from ValuesPerThread 500 to 1000, they are the nearly identical from ValuesPerThread 1000 to 2000, suggesting the configuration has stabilized and will maintain a steady throughput and bandwidth even as the program scales larger. The reduction in performance from ValuesPerThread 500 to 1000 is likely due to diminishing returns of scaling up the work done by individual threads. Ideally, ValuesPerThread and the total problem size would not be tied to the same input variable, allowing ValuesPerThread to be isolated to determine the optimal value for a fixed problem size.

Problem-2: Shared CUDA Matrix Multiply

Q3

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult00 16
Grid Dimensions: 16x16
Data dimensions: 256x256
Grid Dimensions: 16x16
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000033 (sec), nFlops: 33554432, GFlopsS: 1012.499916
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult00 32
Grid Dimensions: 32x32
Data dimensions: 512x512
Grid Dimensions: 32x32
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000127 (sec), nFlops: 268435456, GFlopsS: 2112.382564
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult00 64
Grid Dimensions: 64x64
Data dimensions: 1024x1024
Grid Dimensions: 64x64
Block Dimensions: 16x16
Footprint Dimensions: 16x16
Time: 0.000903 (sec), nFlops: 2147483648, GFlopsS: 2377.824513
```

The main conclusion from these results is that performance with matmultKernel00 begins to plateau once the dimensions of the matrices get larger than 512x512. This is indicated by the near doubling of GFlopS from 1012.500 to 2112.383 when increasing the matrices from 256x256 to 512x512, demonstrating increased throughput. The throughput increased because the program argument increases the grid dimensions along with the data dimensions, making use of more blocks per grid and increasing parallelism. This improvement is also demonstrated by the kernel execution time only increasing 284.848% despite the total compute rising by 700 %, evidenced by nFlops quadrupling as the total elements in the matrices quadrupled.

From 512x512 to 1024x1024, however, GFlopS only increased by 12.566% despite the problem size once again quadrupling. This illustrates the plateau in parallelism, which justifies the increased tile size in the next exercise.

Q4

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult01 8
Grid Dimensions: 8x8
Data dimensions: 256x256
Grid Dimensions: 8x8
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000023 (sec), nFlops: 33554432, GFlopsS: 1450.901942
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult01 16
Grid Dimensions: 16x16
Data dimensions: 512x512
Grid Dimensions: 16x16
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000079 (sec), nFlops: 268435456, GFlopsS: 3391.264780
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-A$ ./matmult01 32
Grid Dimensions: 32x32
Data dimensions: 1024x1024
Grid Dimensions: 32x32
Block Dimensions: 16x16
Footprint Dimensions: 32x32
Time: 0.000470 (sec), nFlops: 2147483648, GFlopsS: 4567.545261
```

Increasing the elements in the resulting C block computed by each thread from 1 to 4 is an example of thread coarsening. This strategy was employed in vecaddKernel00, where each thread handled ValuesPerThread elements. This raises the arithmetic intensity, putting less pressure on the bandwidth. In each inner loop iteration, 2 elements each from A and B are loaded into registers and used in 4 multiplications, making better use of registers. Similarly, there are more results being accumulated in registers at once. These improvements were also combined with the grid dimensions being halved compared to matmultKernel00 because of the increased footprint (tile) size.

The kernel execution times from matmultKernel01 are faster than the corresponding execution times of the same sizes from matmultKernel00, with the largest data size of 1024x1024 achieving a speedup of 1.921x with the new kernel. Additionally, the GFlopS substantially increasing as the problem size increases demonstrates the kernel is scaling up properly and getting closer to the peak FLOPS.

Q5

As with all HPC, a key rule to maximize throughput and decrease execution time is to increase the arithmetic intensity. Better arithmetic intensity was also achieved in both in matmultKernel01 through thread coarsening by reusing registers. Similarly, configuring larger enough tile sizes to fully take advantage of shared memory was shown to be very important in the superior execution time and throughput of matmultKernel01 compared to matmultKernel00. The improvement in execution time from vecaddKernel00 to vecaddKernel01 also demonstrated the importance of coalescing memory, which improved bandwidth. Another clear rule is to maximize hardware occupancy. Both experiments Q3 and Q4 illustrated this by their increasing GFlopS as the problem size increased.

Part-B CUDA Unified Memory

Q1

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q1 1
Compute time (s): 0.001953
CPU verify PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q1 5
Compute time (s): 0.009063
CPU verify PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q1 10
Compute time (s): 0.017489
CPU verify PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q1 50
Compute time (s): 0.086091
CPU verify PASSED
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q1 100
Compute time (s): 0.174514
CPU verify PASSED
```

As expected, execution time on the CPU increases proportionally to the problem size. This linear relationship is demonstrated by the execution time increasing by 8.95x when increasing the problem size by 10x (from 1 to 10 million), and then again as execution time increasing by 9.98x when the problem size increases by 10x (from 10 to 100 million). This illustrates the sequential nature of the CPU.

Q2

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q2 1
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.041305 2=0.000601 3=0.000032
Scenario hot times (s): 1=0.041262 2=0.000577 3=0.000013
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q2 5
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.266638 2=0.007033 3=0.000202
Scenario hot times (s): 1=0.332860 2=0.007061 3=0.000233
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q2 10
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.665087 2=0.014111 3=0.000477
Scenario hot times (s): 1=0.667568 2=0.014115 3=0.000502
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q2 50
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=3.356666 2=0.070969 3=0.002525
Scenario hot times (s): 1=3.356588 2=0.070983 3=0.002566
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q2 100
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=6.670520 2=0.140887 3=0.005026
Scenario hot times (s): 1=6.670588 2=0.140865 3=0.005091
```

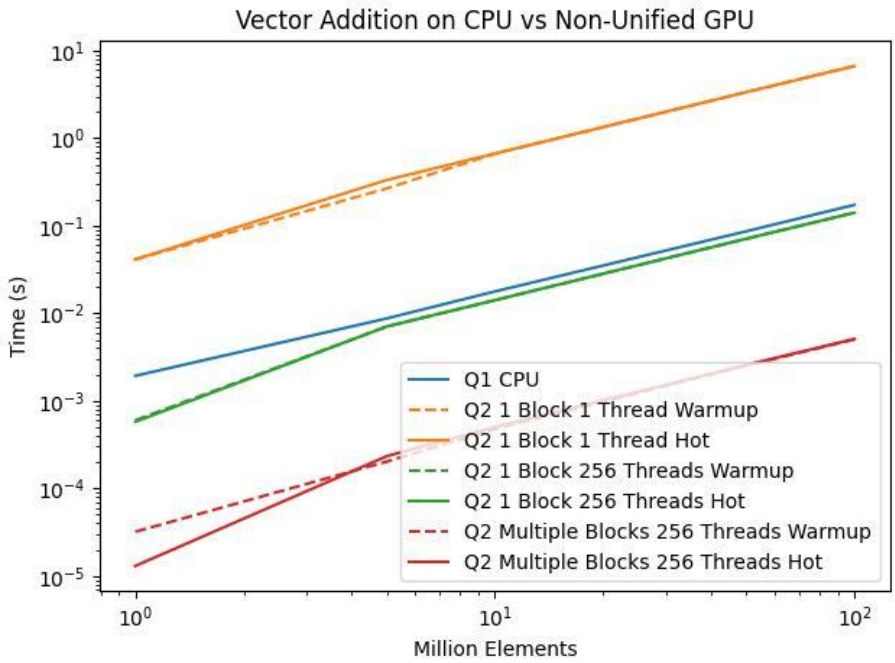
In scenario 1 with only 1 thread doing any work, this implementation performs slower than the CPU. The GPU has not yet taken advantage of any parallelism. With 1 block and 256 threads, this kernel begins to show slight improvements over the CPU implementation with the 100 million element setup only achieving a speedup of 1.239x. This is because limiting the configuration 1 blocks mean only one SM is being used, underutilizing hardware and potential parallelism. Scenario 3 using multiple blocks with 256 threads is the best configuration because it dynamically finds the optimal grid size based on the problem, leveraging more SMs. This results in a speedup of 27.669x from scenario 2 to scenario 3 on 100 million elements. The warmup and hot times are extremely similar across all scenarios.

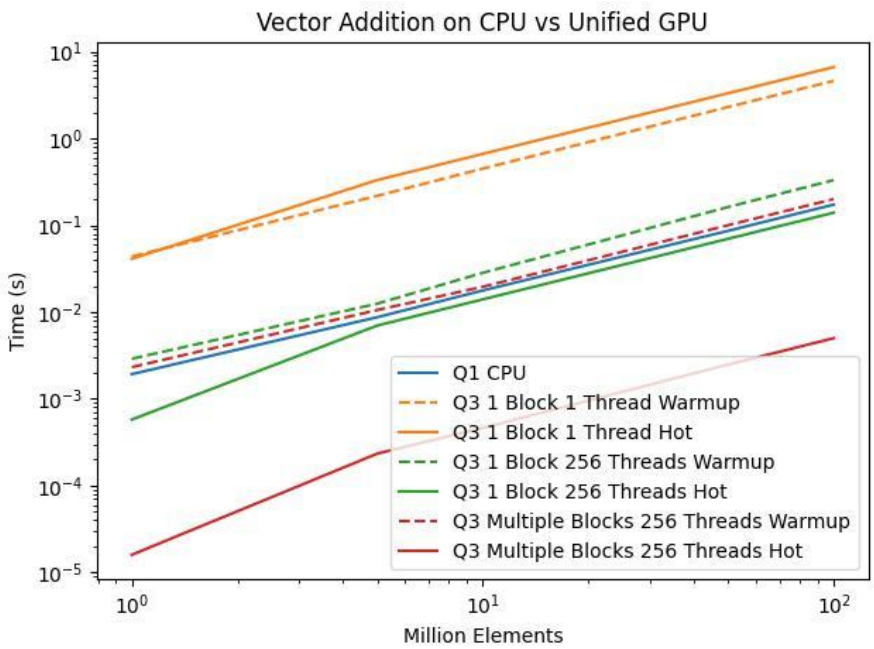
Q3

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q3 1
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.043749 2=0.002899 3=0.002321
Scenario hot times (s): 1=0.041136 2=0.000578 3=0.000016
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q3 5
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.218390 2=0.012505 3=0.010558
Scenario hot times (s): 1=0.331891 2=0.006994 3=0.000234
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q3 10
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=0.450181 2=0.028340 3=0.019570
Scenario hot times (s): 1=0.664482 2=0.014020 3=0.000463
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q3 50
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=2.302808 2=0.162423 3=0.100268
Scenario hot times (s): 1=3.322097 2=0.069921 3=0.002486
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-B$ ./q3 100
Scenario 1 verify PASSED
Scenario 2 verify PASSED
Scenario 3 verify PASSED
Scenario warmup times (s): 1=4.620120 2=0.331791 3=0.200413
Scenario hot times (s): 1=6.643738 2=0.139728 3=0.005009
```

When comparing hot kernel execution times, unified memory achieved nearly identical performance to non-unified memory across all scenarios. However, the main goal of using unified memory is for ease of development, which was observed as c3.cu no longer needed to create both sets of CPU and GPU pointers and use cudaMemcpy before and after kernel execution. Since only the kernel execution times were measured, the speedup of memory management before and after kernel execution cannot be directly compared. However, the increase in warmup times in q3 compared to q2 indicates that the on-demand paging was slowing the kernel.

Q4





Both plots show very similar trends because the hot kernel execution times are nearly identical between Q2 and Q3. Both charts show the CPU implementation (blue) is faster than the 1 block 1 thread implementation (orange) and sometimes faster than the 1 block 256 thread implementation (green). The largest difference between plots is the gap between warmup and hot when using unified memory that is not present without unified

memory. Again, this is because of the page faulting that occurs when the data is first touched when using unified memory, which occurs during the warmup run. This warmup is so long in unified memory that the Multiple Blocks 256 Threads Warmup is slightly slower than the CPU, although the Multiple Blocks 256 Threads Hot is still just as fast as without unified memory.

Part-C: Convolution in CUDA

C1-C3

```
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-C$ ./c1
Launching ConvKernel with grid (64,64,64) and block (16,16,1)
ConvKernel time: 7.940 ms
Checksum computed by GPU: 122756344698240.000000000000
Checksum OK (within tolerance)
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-C$ ./c2
Launching ConvKernel with grid (64,64,64) and block (16,16,1)
ConvKernel time: 7.949 ms
Checksum computed by GPU: 122756344698240.000000000000
Checksum OK (within tolerance)
(base) as7629@cuda-l2-instance-20260322-004309:~/CUDA1/Part-C$ ./c3
cuDNN output dimensions: N=1, C=64, H=1024, W=1024
cuDNN chose algorithm: 0
Total time of cudnnConvolutionForward: 19.105 ms
Checksum computed by GPU: 122756344698240.000000000000
Checksum OK (within tolerance)
```

Individual Program Analysis

C1 follows the same structure as matmultKernel00, where each thread computes one element of the resulting convolved output. This was achieved by storing the input tensor A in a DMatrix with (channels * H) rows and (W) columns. Similarly, the kernel was fit into a DMatrix with (K * channels * H) rows and (W) columns. Each thread accumulates the partial output during each of the 3 channels, having the side-effect of summing across channels without an explicit instruction for this.

C2's improvements using tiling was tricky to implement because – due to padding -- the input tensor height and width (18x18) do not match the block size (16x16), which is needed to continue having each thread compute a single element of the output. Similarly to matmultKernel00, threads don't necessarily load the elements that they will later use in computing the output. This necessitated the complex looping for loading in the tensor A to shared memory, which was coalesced to have adjacent threads load from adjacent locations in A's memory.

The implementation of C3 was greatly aided by a TA's post (#226) on Ed, which suggested what to do when CUDNN CONVOLUTION FWD PREFER FASTEST is not supported. I had cuDNN find the fastest convolution algorithm using cudnnFindConvolutionForwardAlgorithm, which runs microbenchmarks with my given configuration of tensors and shapes. I then use the fastest algorithm with cudnnConvolutionForward. To keep the rest of my main function consistent with c1.cu and c2.cu, I passed in the input tensor already padded, which necessitated defining padding as 0 within the kernel. I also relied on code snippets from lecture notes for querying and allocating the workspace.

Results analysis:

The output shown above is from all 3 programs (also documented in program_output.csv). C1 and C2 performed incredibly similarly, with a difference in kernel execution time of .009 s. Although tiling and shared memory usage are meant to improve performance, they also added overhead, especially due to the complex logic loading the input tensor into shared memory. Additionally, the kernel is so small that it was likely being cached into L1 or L2 during C1 since they were used by multiple threads and blocks, meaning that shared memory wasn't reducing as many loads from global memory.

C2 performed 2.403x faster than C3. One major reason for this could have been because there were no warmup kernel runs during these experiments. This could have disproportionately impacted C3 if cuDNN did more setup during its first run. Additionally, cuDNN is not optimized for double precision, which is rare in deep learning use-cases that cuDNN is typically used for. Similarly, cuDNN is optimized for batched training, whereas this setup only had one batch and 3 channels, minimizing the potential parallelization advantage of cuDNN.